



Knight's Blvd Retail Management System – V1

Project Overview

Abstract

Knight's Blvd Retail Management System is a Python-based local retail management application developed to manage core retail operations such as products, customers, orders, inventory, and sales records.

Unathi Ndlela
Insane.misfit1@gmail.com

The application uses Python for its application logic and SQLite for persistent data storage.

As part of the V1 development process, I performed structured software testing to verify that the application's core functionality worked as expected, identify defects, document their impact, and confirm that corrections were successful through retesting.

The QA process included functional testing, negative testing, database persistence testing, regression testing, defect tracking, evidence collection, and retesting.

01. QA Outcome

The V1 test cycle covered 15 test cases.

- 15/15 test cases passed after corrections
- 5 defects identified
- 5 defects fixed
- 5 retests completed
- 5/5 retests passed
- 15 evidence records documented

The project demonstrates my ability to approach a software application from a QA perspective: not simply checking whether features work, but deliberately testing expected and unexpected behavior, documenting defects, verifying fixes, and maintaining traceable test evidence.

02. My QA Role & Responsibilities

For the Knight's Blvd V1 project, I approached the application from a software quality assurance perspective. My focus was to verify that the system behaved as expected, identify problems through testing, document defects, and confirm that corrections resolved the original issues.

My Responsibilities

- Designed structured test cases covering the application's main retail functions.
- Executed functional test for products, customers, orders, inventory, and sales.

- Performed negative testing using invalid quantities, duplicate products, and insufficient-stock scenarios.
- Tested database persistence by restarting the application and verifying that saved information remained available.
- Reproduced application errors and recorded the conditions under which they occurred.
- Documented expected and actual results for failed or problematic scenarios.
- Recorded defects with severity, reproduction steps, root cause, and corrective action.
- Performed regression testing after fixes were implemented.
- Retested previously failed scenarios to verify that the defects have been resolved.
- Maintained an evidence index linking screenshots to test cases, defects, and retests.
- Recorded the overall test results and prepared a QA test summary.

QA Approach

My approach followed a simple traceable cycle:

Test → Observe → Record → Investigate → Fix → Retest → Verify

This allowed me to move beyond simply checking whether the application worked and instead evaluate how it behaved under different conditions and whether previously identified problems were genuinely resolved.

03. Testing Scope

The testing scope for Knight's Blvd V1 focused on the core functionality required for a local retail management system.

In Scope

Product Management

- Viewing products and inventory.
- Adding new products.
- Searching for products.

- Updating existing products information.
- Remove products.
- Verifying product changes persist after application restart.
- Handling duplicate product entries.

Customer Management

- Adding customer records.
- Viewing saved customer records.
- Verifying customer information can be retrieved from the database.

Order & Inventory Management

- Placing customer orders.
- Calculating order quantities and totals.
- Deducting inventory after successful orders.
- Preventing orders when requested quantities exceed available stock.
- Rejecting invalid order quantities.

Sales Management

- Recording completed sales.
- Retrieving and displaying sales records.

Database & Application Behavior

- Verifying data persistence using the SQLite database.
- Testing application behavior after restarting.
- Verifying normal application exit.
- Investigating database-related errors.

Out of Scope

The following areas were not included in the V1 testing cycle:

- Online payment gateway integration.

- Production deployment.
- Multi-user or concurrent access.
- Penetration/security testing.
- Large-scale performance and load testing.
- Cloud/database hosting.
- Automated test frameworks.
- Integration with external retail platforms.

These areas may be considered in future versions as the application becomes more advanced.

Scope Summary

The objective was not to test every possible scenario that could exist in a production retail platform. The objective was to establish that the core V1 functionality works reliably under the tested conditions and that identified defects can be tracked, corrected, and verified.

04. Test Strategy

The Knight's Blvd V1 testing approach combined several testing techniques to evaluate both normal application behavior and how the system responded to invalid or unexpected conditions.

4.1 Functional Testing

Functional testing was used to verify that the application's features performed their intended functions.

Examples included:

- Viewing products.
- Adding products.
- Searching for products.
- Updating products.
- Removing products.
- Adding and viewing customers.
- Processing orders.

- Viewing sales records.

Each function was tested against a defined expected result and the actual application behavior or crashes.

4.2 Negative Testing

Negative testing was used to determine how the application responded to invalid inputs and conditions.

Examples included:

- Entering a quantity of 0.
- Entering a non-numeric quantity.
- Requesting more stock than was available.
- Attempting to add a duplicate product.

The objective was to confirm that invalid conditions were handled safely rather than causing unexpected application behavior or crashes.

4.3 Database Persistence Testing

Because the application uses SQLite for data storage, persistence was specifically tested.

After creating, updating, or removing records, the application was restarted and the relevant records were checked again.

This helped verify that changes were saved to the database rather than existing only temporarily during the current application session.

4.4 Regression Testing

Regression testing was performed after defects were corrected.

Previously problematic scenarios were executed again to determine whether:

1. The original defect had been resolved.
2. The intended functionality now worked correctly.
3. The correction had not introduced another obvious problem in the tested area.

Five identified defects were retested, with all five retests passing.

4.5 Defect-Based Testing

When an error occurred, the scenario was documented rather than simply moving on.

The defect process included:

Reproduce → Record → Investigate → Correct → Retest

Examples included a closed database connection error, a tuple/object mismatch during product searching, and problems with sales handling.

4.6 Evidence-Based Testing

Screenshots and evidence records were maintained throughout the testing process.

Each evidence record was linked to relevant test cases and, where applicable, defects and retests.

This created traceability between the testing activity and the final QA documentation.

4.7 Overall Strategy

The overall strategy was focused on establishing confidence in the core V1 functionality rather than claiming complete production-level coverage.

The testing process therefore prioritized:

Core functionality + invalid inputs + data persistence + defect investigation + regression verification.

5.Defects Found

During the V1 testing cycle, five defects or validation issues were identified. Each issue was documented, corrected, and retested to verify the result.

BUG-001 – Closed Database Connection

Related Test Case: TC-015

Severity: High

Status: Fixed

Retest: RET-001 – Passed

Problem

The application attempted to perform a database operation after the database connection had already been closed.

Observed Result

The application produced:

`sqlite3.ProgrammingError: Cannot operate on a closed database.`

Root Cause

Obsolete application code remained after the menu system had been updated. Th at code attempted to access the database after the connection had been closed.

Verification

The application was executed again and exited normally without producing the closed-database error.

BUG-002 – Product Search Tuple/Object Mismatch

Related Test Case: TC-003

Severity: Medium

Status: Fixed

Retest: RET-002 – Passed

Problem

The database returned product records as tuples, while the search functionality attempted to access them as Product objects.

Observed Result

The application produced:

AttributeError: 'tuple' object has no attribute 'name'

Root Cause

There was a mismatch between the data structure returned by the database layer and the way the search function attempted to access the product data.

Correction

The search and product-display logic was adjusted to use the appropriate tuple indexes returned by the database.

Verification

Product searches were executed again, and matching product information was displayed correctly.

BUG-003 – Sales Display/Handling Error

Related Test Case: TC-013

Severity: High

Status: Fixed

Retest: RET-003 – Passed

Problem

The sales display functionality incorrectly handled fields from the stored sales records, resulting in incorrect or missing values during processing.

Observed Result

The sales operation initially produced an error because sales fields were being handled incorrectly.

Debugging showed transaction information including the customer, product, quantity, and total, which helped identify the data-handling issue.

Correction

The sales data handling and display logic was corrected so that the appropriate fields from the stored sales records were used.

Verification

The View Sales function was executed again and the stored sales records displayed successfully.

BUG-004 – Duplicate Product Handling

Related Test Case: TC-002

Severity: Medium

Status: Fixed

Retest: RET-004 – Passed

Problem

Testing duplicate product entry exposed an issue in the duplicate-product handling flow.

Observed Result

An existing product such as Black Cargo Pants was correctly identified as already existing, but the surrounding handling required correction to ensure the application rejected the duplicate safely without crashing.

Correction

The duplicate-product handling was corrected so that an existing product is corrected so that an existing product is rejected with a clear message instead of rejecting with a clear message instead of allowing the application to fail unexpectedly.

Verification

The duplicate product scenario was executed again. The duplicate was rejected safely and the application continued operating normally.

BUG-005 – Invalid Quantity Validation

Related Test Case: TC-012

Severity: Low

Status: Fixed

Retest: RET-005 – Passed

Problem

Order quantity validation required correction to properly handle invalid use input.

Test Conditions

Two invalid inputs were tested:

- Quantity of 0
- Non-numeric quantity

Expected Behavior

The application should reject invalid quantities and continue operating without crashing.

Result

The application correctly rejected 0 with a message indicating that the:

Quantity must be greater than zero.

Non-numeric input with:

Invalid quantity. Please enter a number.

Verification

Both scenarios were retested successfully.

Defect Summary

Severity	Defects
High	2
Medium	2
Low	1
Total	5

All five identified defects were corrected and subsequently retested.

Retest result: 5/5 passed.

6. Fix & Retest Process

After identifying defects during testing, I followed a structured process to verify that corrections resolved the original problems.

QA Defect Cycle

The process followed was:

Test → Identify Defect → Reproduce → Document → Correct → Retest → Verify

1. Identify

A test case produced behavior that differed from the expected result.

2. Reproduce

The same steps were performed again to confirm that the problem could be reproduced consistently.

3. Document

The issue was recorded in the Defect Log, including:

- Defect ID
- Related test cases
- Steps to reproduce
- Expected result
- Actual result
- Severity
- Root cause
- Fix applied
- Retest requirement

4. Correct

The relevant application logic was investigated and corrected

5. Retest

The original test scenario was executed again after the correction.

6. Verify

The new result was compared against the expected behavior. Evidence was recorded where applicable.

Retest ID	Defect	Test Area	Result
RET-001	BUG-001	Application Exit	Pass
RET-002	BUG-002	Product Search	Pass
RET-003	BUG-003	Sales Display	Pass
RET-004	BUG-004	Duplicate Product Handling	Pass
RET-005	BUG-005	Quantity Validation	Pass

Retest Outcome

All 5 identified defects were retested successfully.

This provided evidence that the corrections addressed specific issues identified during the V1 testing cycle.

Traceability

The QA documentation maintains a relationship between maintains a relationship between the original test, defect, correction, retest, and evidence.

For example:

TC-003 → BUG-002 → FIX RET-002 → EV-003

This traceability makes it possible to follow a defect from its initial discovery through to verification of the correction.

The same approach was applied across the identified defects where supporting evidence was available.

7. Test Evidence

Evidence was collected throughout the Knight's Blvd V1 testing cycle to provide a visual record of application behavior.

Each evidence item was assigned to a unique Evidence ID and linked to the relevant test case. Where a test involved a defect and subsequent retest, the evidence was also linked to the relevant defect or retest.

Evidence Coverage

The Evidence Index contains 15 evidence records, covering the main V1 testing areas.

Evidence	Testing Area
EV-001	Main application menu
EV-002	Product inventory
EV-003	Product search
EV-004	Product update
EV-005	Update persistence after restart
EV-006	Product removal
EV-007	Removal persistence after restart
EV-008	Customer creation
EV-009	Customer records
EV-010	Successful order and receipt
EV-011	Insufficient stock handling
EV-012	Invalid quantity validation
EV-013	Sales display
EV-014	Database persistence
EV-015	Normal application exit

Evidence Traceability

The evidence was organized so that a reviewer can trace a testing activity back to its supporting record.

For example:

TC-003 → BUG-002 → RET-002 → EV-003

This shows the relationship between the original product-search test, the defect that was identified, the subsequent retest, and the supporting evidence.

Other evidence records document successful functionality and persistence scenarios, including product management, customer management, order processing, inventory validation, sales records, and database behavior.

Screenshot Management

The Evidence Index contains a dedicated field for the screenshot or file name associated with each Evidence ID.

Where an actual screenshot filename has not yet been assigned, the field is intentionally left blank rather than creating a filename that does not exist.

This maintains accurate traceability between the QA documentation and the underlying project files.

Evidence Purpose

The evidence serves three main purposes:

1. Verification – demonstrates observed application behavior.
2. Traceability – connects screenshots to specific test cases and defects.
3. Portfolio Demonstration - provides visual proof of the testing activities performed during the project.

The evidence should be viewed together with the Test Cases, Test Execution, Defect Log, and Retest Results rather than as standalone screenshots.

8.Final Test Results

The Knight's Blvd V1 testing cycle covered the applicant's core retail management functionality.

A total of 15 planned test cases were executed. During testing, 5 defects were identified and documented. Each identified defect was corrected and subsequently retested.

Test Results

Metric	Result
Test Cases Executed	15
Test Cases Passed After Fixes	15
Test Cases Failed at Final Verification	0
Defects Identified	5
Retests Performed	5
Retests Passed	5
Evidence Records	15

Functional Coverage

The testing cycle covered:

- Product management
- Product search
- Product updates and removal
- Customer management
- Order processing
- Inventory validation
- Quantity validation
- Sales records
- SQLite database persistence
- Application exit behaviour

Defect Resolution

The five identified defects covered several areas of the application, including database connection handling, product search data handling, sales processing, and quantity validation.

Following corrections, each defect was retested against its original scenario.

5/5 retests passed.

Final V1 Assessment

Based on the test cases executed and the defects identified and retested, the tested core functionality of Knight's Blvd V1 was operating as expected at the end of the testing cycle.

This result applies specifically to the documented V1 test scope and test conditions. It does not represent a claim that every possible production scenario, security condition, performance condition, or future feature has been tested.

QA Deliverable

The testing cycle produced a structured QA evidence package consisting of:

Test Plan → Test Cases → Test Execution → Defect Log → Retest Results → Evidence Index →
Test Summary

Together, these records provide traceability from the original test through defect discovery, correction, retesting, and final verification.

9.Lessons Learned

The Knight's Blvd V1 project gave me practical experience in approaching software from a quality assurance perspective.

Testing Beyond the Happy Path

I learned that testing a feature only when everything goes correctly is not enough. Testing invalid quantities, duplicate products, insufficient stock, and unexpected application conditions revealed issues that normal usage did not expose.

Understanding Errors

When the application produced errors, I learned to investigate what the error was indicating rather than simply treating it as a failure.

For example, the tuple/object mismatch in product searching showed me how differences between database output and application logic can affect functionality.

Important of Database Persistence

Testing the application after restarting showed me the importance of verifying that information is stored and retrieved correctly.

A feature appearing to work during one session does not necessarily prove that its data has been saved correctly.

Retesting After Fixes

I learned that fixing a defect is only part of the process. The original scenario needs to be executed again to verify that the correction solved the problem.

This helped me understand the relationship between defect fixing and regression testing.

Importance of Documentation

Maintaining Test Cases, Test Execution records, a defect log, retest results, and an Evidence Index showed me the value of structured QA documentation.

The documentation creates traceability and makes it possible for another person to understand what was tested, what happened, what was corrected, and how the correction was verified.

Developing a QA Mindset

The project helped me move from simply asking:

“Does the application work?”

To asking:

“How does the application behave under different conditions, and how can I demonstrate that behaviour with evidence?”

That shift is one of the most important lessons I gained from the project.

10. Project & Portfolio Summary

Project Summary

Knight's Blvd Retail Management System V1 is a Python and SQLite-based retail management application created to manage core retail operations including products, customers, inventory, orders, and sales records.

The project was also used as practical software test exercise, allowing me to develop and demonstrate QA skills through hands-on testing of a working application

QA Summary

During V1 testing cycle, I:

- Created and executed 15 test cases.
- Tested core retail functionality and invalid input scenarios.
- Identified and documented 5 defects.
- Investigated the causes of the identified issues.
- Verified corrections through 5 retests.
- Achieved 5/5 passed retests.
- Maintained 15 evidence records.

Created structured QA documentation covering planning, execution, defects, retesting, evidence, and results.

Skills Demonstrated

The project demonstrates practical experience with:

- Software Testing
- Functional Testing
- Negative Testing
- Regression Testing
- Database persistent testing
- Test execution
- Defect management
- Defect identification
- Reproduction
- Expected vs. actual result analysis
- Severity classification
- Root-cause investigation
- Fix verification
- QA Documentation
- Test planning
- Test case design
- Test execution records
- Defect logging
- Retest documentation
- Evidence traceability

- Test reporting
- Technical Environment
- Python
- SQLite
- Local application testing
- Database-driven application behaviour

Portfolio Package

The Knight's Blvd QA project can be presented as a complete portfolio package containing:

- Knight's Blvd application/source code
- QA Portfolio Case Study
- Application screenshots
- Test and defect evidence

The workbook provides detailed testing records, while the case study explains the testing process and results in a format that can be quickly understood by a client or potential employer

Closing Statement

Knight's Blvd demonstrates my ability to take a functioning software project, approach it from a QA perspective, test its behaviour under different conditions, identify and document defects, verify corrections, and organise the results into professional testing documentation.

The project represents a practical foundation for continuing to develop my software testing skills through additional projects, broader test coverage, and more advanced QA techniques.